

Agenda

- Multer
- Typescript
- Datatypes

Multer

- Multer is a node.js middleware for handling multipart/form-data, which is primarily used for uploading files.

```
const express = require('express')
const multer = require('multer')
const upload = multer({ dest: 'uploads/' })

const app = express()

app.post('/profile', upload.single('avatar'), function (req, res, next) {
  // req.file is the `avatar` file
  // req.body will hold the text fields, if there were any
})

app.post('/photos/upload', upload.array('photos', 12), function (req, res, next) {
  // req.files is array of `photos` files
  // req.body will contain the text fields, if there were any
})
```

Introduction

- TypeScript is JavaScript with syntax for types
- Its a Better JavaScript, a Superset of JavaScript
- Using TypeScript compiler JavaScript code can be compiled
- Developed by Microsoft
- Features
 - Strongly typed
 - interface
 - decorators
 - generics
 - namespace
 - Object Oriented Programming Language

Why Typescript

1. Static typing
2. improved code maintainability
3. advanced features

Installation

1. Download the node
2. install the typescript compiler using npm

```
npm install -g typescript
```

3. check the version

```
tsc --v
```

Execution

1. Create a Hello.ts file
2. compile using below command

```
tsc Hello.ts
```

3. execute the created js file

```
node Hello.js
```

Datatype

- Their are 5 categories of datatypes

1. Primitive (same as JS)

- number
- string
- boolean
- null
- undefined
- bigint
- symbol

2. Object

- Array
- Tuple
- Object

3. Special

- any
- unknown
- void

- never

4. Advanced

- type alias
- union
- intersection
- enum
- literal

5. Function

Special DataTypes

1. void

- void is generally used as the return type of a function that does not return a meaningful value.
- A function with void return type can still technically execute a return; statement.
- It is most commonly used for functions that perform an action but don't return a result.
- void does not mean that the function can never return.
- It means that the caller should not expect a useful return value.

```
```ts
function printMessage(): void {
 console.log("Hello TypeScript");
}
```
```

2. never

- never represents a value that will never occur.
- It is commonly used as the return type of functions that never successfully complete.
- Typical examples are:
 - Functions that always throw an error.
 - Functions containing an infinite loop.

```
```ts
function throwError(message: string): never {
 throw new Error(message);
}

function infiniteLoop(): never {
 while (true) {
 console.log("Running...");
 }
}
```
```

3. any

- any disables most of TypeScript's type checking for that value.
- It can be used for:
 1. Variables
 2. Function parameters
 3. Function return types
 4. Properties, etc.
- A variable of type any can contain any type of value.

```
``ts
let value: any;
value = 10;
value = "Hello";
value = true;
value = { name: "Rohan" };
``
```

- we can also perform operations on an any value without TypeScript checking whether those operations are valid.
- The second statement may cause a runtime error, because TypeScript is not protecting you from incorrect operations on any.

```
let value2: any = "Hello";
console.log(value2.toUpperCase()); // Valid
console.log(value2.foo.bar);      // TypeScript allows it
```

4. unknown

- it is similar to the any but with more type safe
- unknown can also hold a value of any type.
- However, unlike any, TypeScript does not allow you to perform arbitrary operations on an unknown value.
- You must first establish what type the value has using type checking, type guards, or type assertions.

```
``ts
let value: unknown = "Hello";

//console.log(value.toUpperCase()); // NOT OK

if (typeof value === "string") {
  console.log(value.toUpperCase()); // OK
}
``
```

any vs unknown

- `any` → TypeScript performs little or no type checking.
- `unknown` → TypeScript requires the type to be checked before most operations.

Object Type

1.Array

- An Array is a collection of multiple values stored in a single variable.
- In TypeScript, an array can be defined with a specific data type for its elements.
- All elements of an array generally follow the specified type.

```
let numbers: number[] = [10, 20, 30, 40];
let names: string[] = ["John", "Alice", "Bob"];

// Alternative Syntax
let numbers: Array<number> = [10, 20, 30];
let names: Array<string> = ["John", "Alice"];
```

SUNBEAM INFOTECH